SmartGate IoT

Práctica 2

Middleware IoT y servidor backend en tiempo real (FastAPI + MQTT + WebSockets)

CETI Colomos
Ingeniería en Desarrollo de Software
Internet de las Cosas | Grupo 7P

Lizeth Alexandra Esquivies Torres | Registro 23310409
Caleb Berumen Estrada | Registro 23310408
Docente José Florentino Chavira Sánchez

Fecha: 25 de octubre de 2026

Objetivo y avance

En la semana 1 armamos el nodo sensor de la cochera: un ultrasónico HC-SR04 vigila la entrada, un sensor láser VL53L0X (ToF) vigila el lugar de estacionamiento, un reed switch con imán confirma que la puerta cerró y un servo SG90 mueve la hoja. En esta práctica conectamos ese nodo a un middleware: la telemetría multivariable viaja por MQTT hasta un broker Mosquitto, un backend asíncrono en FastAPI la valida y guarda, y un WebSocket la empuja en tiempo real a una app móvil Android.

La app muestra si la puerta está cerrada, abriéndose, abierta o cerrándose; si hay coche dentro; y las lecturas del ultrasónico y del ToF. Además lanza tres notificaciones: «Se está abriendo la cochera», «La cochera está abierta» y «Se cerró la cochera». Todo se probó con la maqueta real en un ciclo completo de llegada, apertura, estacionamiento y cierre.

Requisitos de la práctica

Requisito	Cómo se cumplió	Estado
Firmware con MQTT y JSON multivariable	La ESP32 arma un JSON con 5 variables cada 500 ms; el gateway ESP32-C3 lo publica por MQTT con QoS 0 o 1.	Cumplido
Backend FastAPI asíncrono	Suscripción con paho-mqtt, validación Pydantic, SQLite y endpoint /stats.	Cumplido
Canalización por WebSockets	Endpoint /ws/telemetria; lo consumen la app Android y un cliente en el navegador.	Cumplido
Experimento 1: QoS 0 vs QoS 1	4 corridas de 90 s: QoS 0 y 1 con red normal y con red degradada.	Cumplido
Experimento 2: stream WebSocket	Cliente en el navegador recibiendo una trama cada 500 ms (figura 8).	Cumplido
Capturas de evidencia	Consolas de la ESP32, del gateway y de FastAPI; cliente WebSocket; app.	Cumplido
Articulación IEEE	Sección dedicada a Bharani y Parthipan (2026).	Cumplido

Arquitectura del sistema

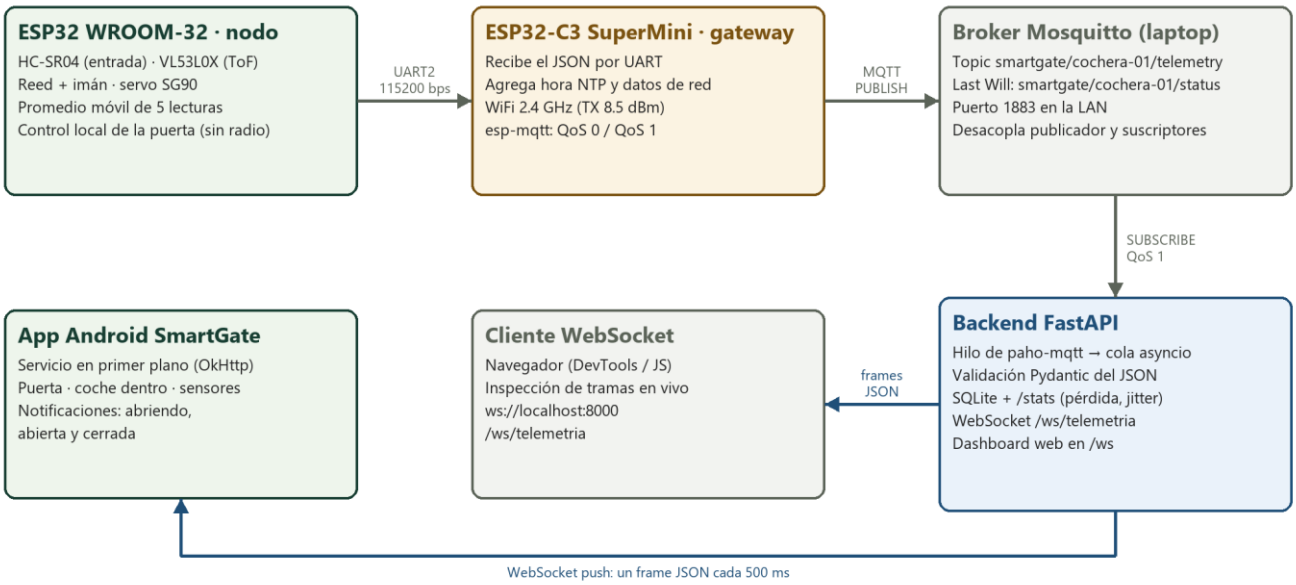


Figura 1. Arquitectura implementada. La flecha de abajo es el push por WebSocket hacia la app.

La guía propone que la ESP32 se conecte sola al WiFi. En nuestra placa eso no es posible: desde la semana 1 comprobamos que la ESP32 WROOM-32 se reinicia por brownout en cuanto enciende la radio (BLE o WiFi, que comparten el mismo bloque de RF). Por eso separamos funciones: la ESP32 sigue leyendo sensores, filtrando y controlando la puerta sin radio, y le pasa el JSON por UART a una ESP32-C3 SuperMini que funciona como gateway de borde y es la que publica en MQTT.

Esta separación no cambia el contrato del middleware: el broker recibe el mismo JSON y el backend no sabe ni le importa cuántas placas hay detrás del topic. También tiene una ventaja que se ve en el experimento 1: el control de la puerta nunca espera a la red.

Implementación

Payload JSON multivariable

Campo	Contenido
node_id, boot_id, sequence_id	Nodo (cochera-01), arranque de la ESP32 y número de mensaje; los huecos de secuencia son paquetes perdidos.
readings.exterior / interior	Ultrasónico y ToF: lectura cruda, promedio móvil de 5 lecturas, validez y motivo («SIN_OBJETO» = libre o vacío).
door	Estado (CERRADA, ABRIENDO, ABIERTA, CERRANDO), modo, reed cerrado y ángulo del servo.
vehicle	Coche detectado ahora por el ToF (inside) y entrada registrada en el ciclo (arrival).
timestamp, network	Agregados por el gateway: hora NTP, RSSI WiFi, QoS usado y duración de publish().

```
{"schema_version":1,"node_id":"cochera-
01","boot_id":"eddbcc9df46f2906","sequence_id":20275,"uptime_ms":10165472,"source":"hardware","filter_window":5,"readings":{"ext
erior":{"raw_mm":40,"filtered_mm":56.4,"valid":true,"sample_count":5,"error":null},"interior":{"raw_mm":null,"filtered_mm":null,
"valid":false,"sample_count":0,"error":"SIN_OBJETO"}},"door":{"state":"ABRIENDO","mode":"AUTO","reed_closed":true,"servo_command
_deg":96},"vehicle":{"inside":false,"arrival":false,"rearmed":false,"exterior_clear":false},"timestamp":1792972050097,"time_sync
ed":true,"network":{"wifi_connected":true,"mqtt_connected":true,"heap_free":178160,"wifi_rssi_dbm":-
63,"qos":1,"publish_us":1508}}
```


Figura 2. Un JSON real recibido por FastAPI (consola del backend, partido en líneas para que quepa).

Nodo ESP32 y gateway ESP32-C3

La ESP32 envía el JSON por UART2 (GPIO17, 115200 bps) hacia el GPIO20 de la C3. El gateway lo valida con ArduinoJson, le agrega la hora NTP y los datos de red y lo publica en smartgate/cochera-01/telemetry con el cliente esp-mqtt de ESP-IDF. Este cliente es asíncrono: publish() solo encola el mensaje y la confirmación PUBACK llega después como evento, sin detener el loop. Al conectarse registra un Last Will en smartgate/cochera-01/status para avisar si el gateway se desconecta.

Un detalle de hardware: la C3 SuperMini no lograba asociarse al router con la potencia de transmisión de fábrica. Al bajarla a 8.5 dBm se conectó con un RSSI de −49 a −55 dBm.

Backend FastAPI y manejo del Event Loop

El cliente paho-mqtt corre en su propio hilo (loop_start). Su callback on_message no toca la base de datos ni los WebSockets: solo pasa el mensaje al Event Loop de asyncio con call_soon_threadsafe y lo deja en una cola. Una tarea asíncrona consume esa cola, valida el JSON con Pydantic (tipos estrictos, rangos y coherencia entre campos), lo guarda en SQLite, actualiza las estadísticas y lo reparte. Cada cliente WebSocket tiene su propia cola de 10 mensajes, así que un teléfono lento no frena la ingesta: si su cola se llena, se descarta su mensaje más viejo.

Endpoint	Uso
WS /ws/telemetria?node_id=cochera-01	Stream de la guía: cada JSON tal como llega por MQTT. Lo usa la app.
WS /ws	Dashboard web: snapshot de estado cada segundo más cada telemetría.
GET /stats y POST /stats/reset	Recibidos, perdidos, % de pérdida, intervalo, jitter y duración de publish().
GET /api/readings, /api/export.csv	Historial guardado en SQLite.

App móvil Android

La app está hecha en Kotlin con Jetpack Compose. Un servicio en primer plano mantiene el WebSocket abierto con OkHttp, se reconecta solo cada 3 s si se cae la red y sigue funcionando con la app cerrada. Cada mensaje actualiza la pantalla; cuando cambia el estado de la puerta, el servicio lanza la notificación correspondiente. El primer mensaje después de conectar solo fija el estado, para no avisar de algo que ya pasó.

Estado de la puerta	Texto en la app	Notificación
CERRADA	Cerrada	«Se cerró la cochera» (al llegar desde otro estado)
ABRIENDO	Se está abriendo	«Se está abriendo la cochera»
ABIERTA	Abierta	«La cochera está abierta»
CERRANDO	Se está cerrando	—

Para «Hay coche dentro» la app combina dos datos: que el ToF vea el carro en ese momento o que el firmware haya registrado su entrada durante el último ciclo. En la prueba real el carro estacionado quedó fuera del haz del ToF; sin la segunda señal la app habría dicho que no había coche.

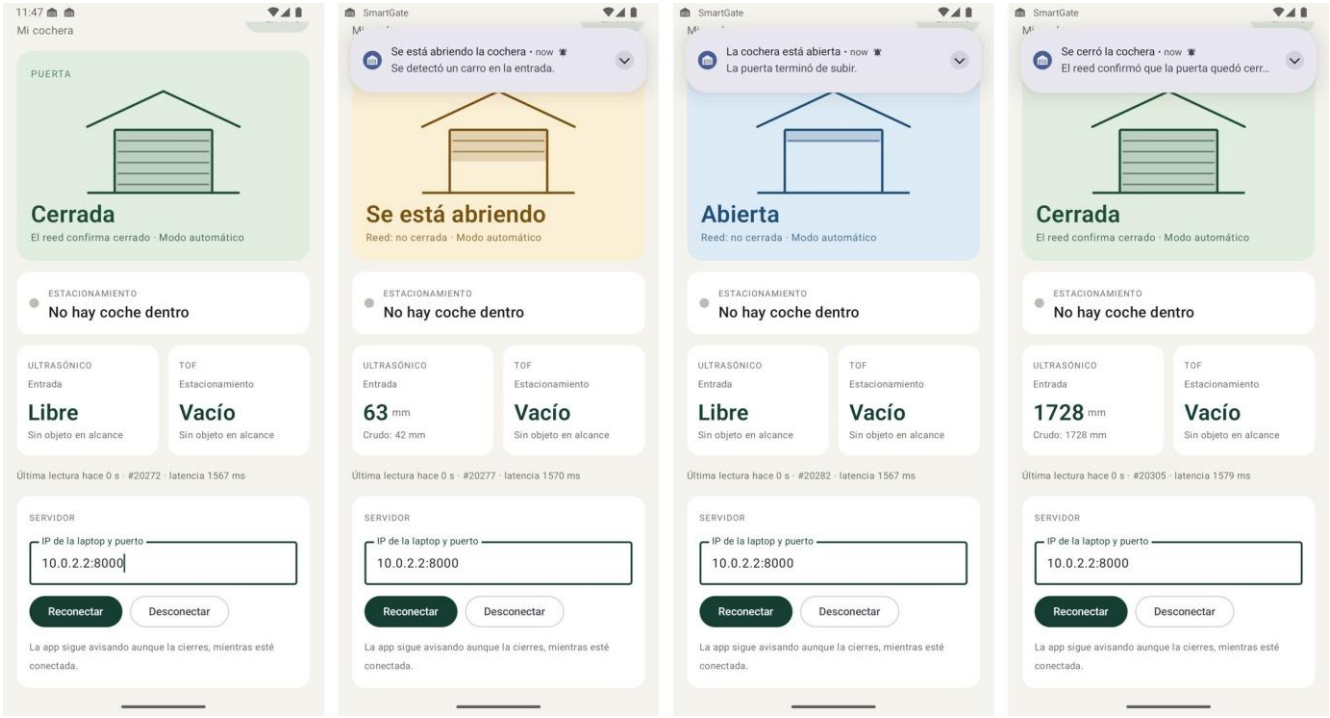


Figura 3. App con la maqueta real, ciclo del 25/10/2026 a las 17:47: cerrada, se está abriendo, abierta y cerrada, con su notificación.

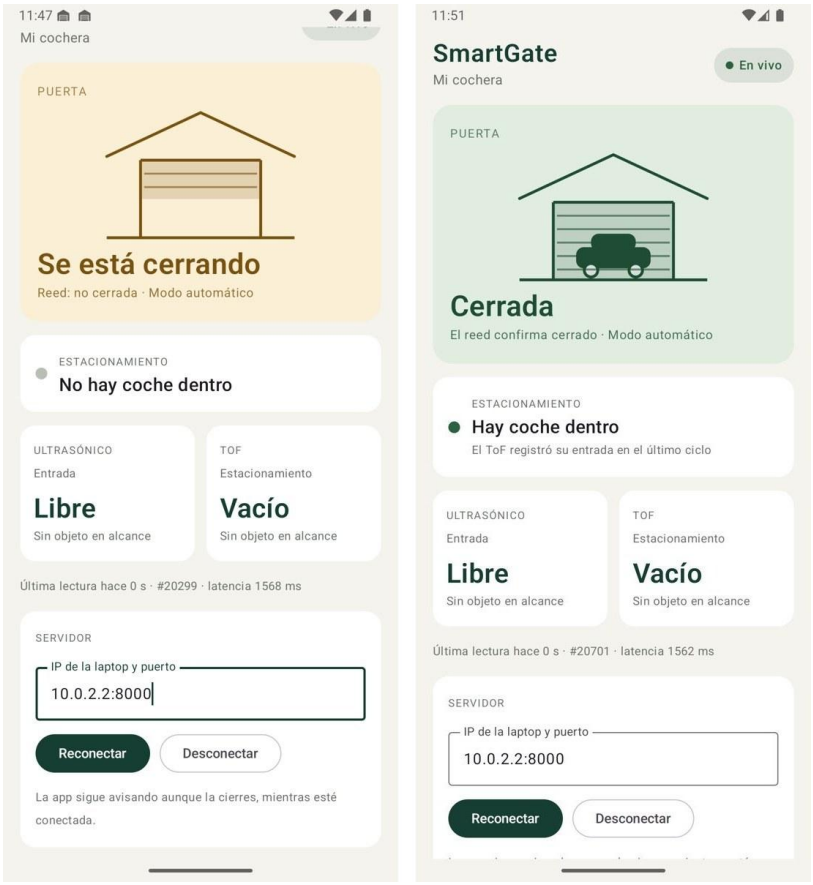


Figura 4. Izquierda: cerrándose. Derecha: después del ciclo, con el coche estacionado.

Evidencias de funcionamiento

Las capturas corresponden al mismo ciclo real: el carro llegó a la entrada (ultrasónico en 56 mm), la puerta abrió, el ToF detectó el carro a 80 mm, la entrada quedó libre y la puerta cerró con el reed confirmado.

Hora	Estado de la puerta	Lectura que lo provocó
17:47:31	ABRIENDO	Ultrasónico a 56 mm durante 3 lecturas: carro en la entrada.
17:47:35	ABIERTA	El servo terminó el recorrido.
17:47:42	CERRANDO	ToF a 80 mm (coche dentro) y entrada libre 2 s.
17:47:46	CERRADA	El reed confirma el cierre.

```
[CERRADA|AUTO] reed=CERRADA | ultra=4000 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
[CERRADA|AUTO] reed=CERRADA | ultra=4000 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
[CERRADA|AUTO] reed=CERRADA | ultra=1850 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
[CERRADA|AUTO] reed=CERRADA | ultra=4000 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
[CERRADA|AUTO] reed=CERRADA | ultra=4000 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
[CERRADA|AUTO] reed=CERRADA | ultra=1846 mm | tof=vacio(fuera de alcance) | carro_dentro=NO | rearmado=SI | servo=90 grados ON
```

Figura 5. Consola serie de la ESP32 de la cochera (COM7, 25/10/2026): una línea por segundo con puerta, reed, ultrasónico, ToF y servo. El JSON sale por UART2 hacia el gateway.

```
17:47:29.619 PUB seq=20271 qos=1 publish=1508 us
17:47:30.168 PUB seq=20272 qos=1 publish=1508 us
17:47:30.716 PUB seq=20273 qos=1 publish=1509 us
17:47:31.149 PUB seq=20274 qos=1 publish=1508 us
17:47:31.698 PUB seq=20275 qos=1 publish=1510 us
17:47:32.140 RESUMEN uart=377 pub=364 ack=365 desc=14 wifi=SI mqtt=SI ultima_linea_hace=456 ms
17:47:32.251 PUB seq=20276 qos=1 publish=1509 us
```

Figura 6. Consola del gateway ESP32-C3: cada PUB es un JSON publicado con QoS 1 y su tiempo de publish(); RESUMEN cada 5 s.

```
17:47:31,133 INFO [MQTT Inbox] {"schema_version":1,"node_id":"cochera-01","boot_id":"eddbcc9df46f2906","sequence_id":2 …}
17:47:31,134 INFO Guardado cochera-01 hardware #20274 latencia=1585 ms
17:47:31,670 INFO [MQTT Inbox] {"schema_version":1,"node_id":"cochera-01","boot_id":"eddbcc9df46f2906","sequence_id":2 …}
17:47:31,670 INFO Guardado cochera-01 hardware #20275 latencia=1568 ms
```

Figura 7. Consola de FastAPI recibiendo el JSON multivariable desde MQTT.

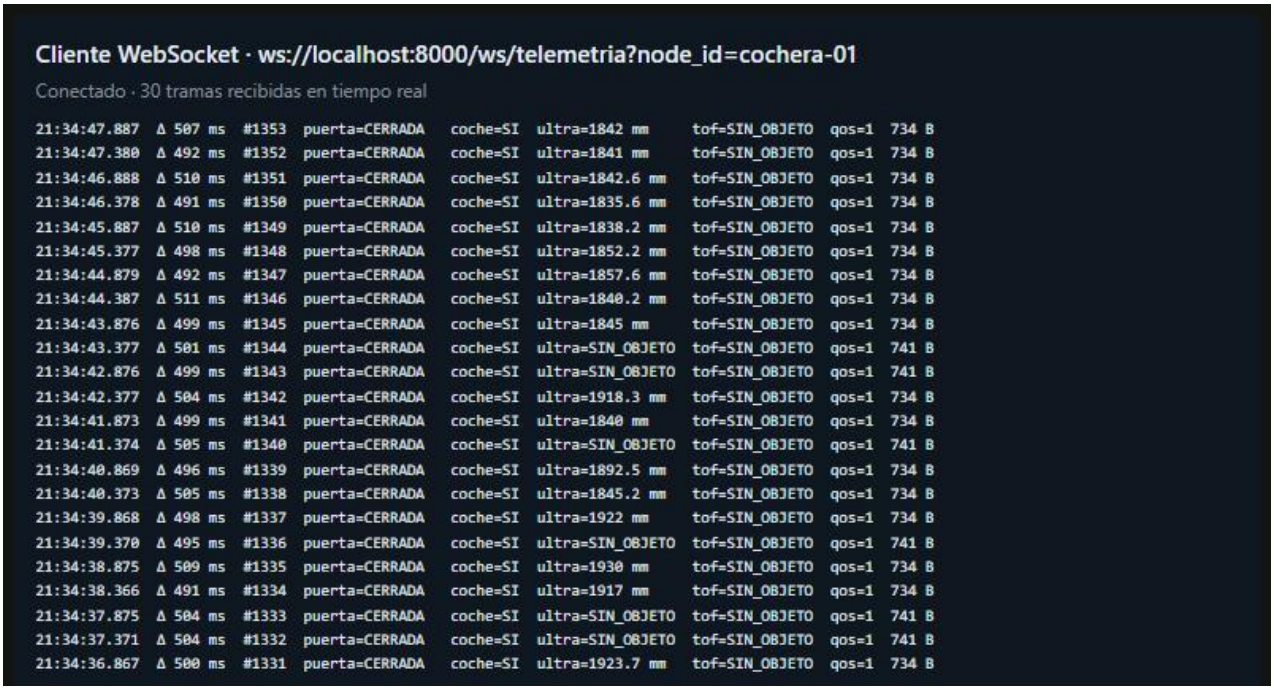


Figura 8. Experimento 2: cliente WebSocket en el navegador conectado a /ws/telemetria, recibiendo tramas en vivo.

Experimento 1: QoS 0 contra QoS 1

Hicimos cuatro corridas de 90 segundos con la maqueta encendida y la puerta cerrada. Para degradar la red sin mover nada, bajamos la potencia de transmisión del gateway de 8.5 dBm a 2 dBm; así la condición se puede repetir. Antes de cada corrida reiniciamos /stats. La latencia es el tiempo entre la hora NTP del gateway y la llegada al backend; como el reloj de la laptop iba adelantado respecto a NTP (1.46 s en las primeras tres corridas y 0.94 s en la última, medido con w32tm), restamos esa diferencia.

Corrida (90 s)	Recibidos / perdidos	Jitter	publish() prom / máx	loop() gateway máx	Latencia prom / máx
QoS 1 · normal	180 / 0	14.5 ms	1.5 ms / 1.7 ms	8.5 ms	85 / 201 ms
QoS 0 · normal	179 / 0	8.3 ms	1.4 ms / 1.7 ms	13.2 ms	144 / 1657 ms
QoS 1 · degradada	179 / 0	347.9 ms	14.1 ms / 2.26 s	2.32 s	190 / 4291 ms
QoS 0 · degradada	180 / 0	326.7 ms	23.8 ms / 4.03 s	4.12 s	140 / 7031 ms

En condiciones normales las dos calidades de servicio se comportaron casi igual: ningún mensaje perdido, un mensaje cada 503 ms y publish() de 1.4 a 1.5 ms. QoS 1 no hizo más lento a publish() porque el cliente esp-mqtt no espera el PUBACK: solo escribe el paquete en el socket y la confirmación llega después como evento (182 publicaciones y 183 confirmaciones, la extra es el mensaje de estado). La diferencia está en lo que pasa si se cae la conexión: con QoS 1 el mensaje queda en la bandeja de salida y se reenvía; con QoS 0 se pierde.

Con la red degradada tampoco se perdieron mensajes, porque MQTT viaja sobre TCP y TCP retransmite lo que no llega. Pero el costo apareció en el tiempo: el jitter subió de 8–15 ms a 327–348 ms, la latencia máxima pasó de 0.2–1.7 s a 4–7 s y publish() llegó a quedarse bloqueado 2.3 s con QoS 1 y 4.0 s con QoS 0. Ese bloqueo no viene del PUBACK sino de que el buffer de envío de TCP se llena mientras espera retransmisiones, así que afecta a ambas calidades de servicio. La pérdida de QoS 0 aparecería con una desconexión real, que en estas corridas no ocurrió.

La latencia es aproximada (±50 ms) porque compara el reloj NTP del gateway con el de la laptop, corregido con el desfase medido. Dos corridas se repitieron porque la ESP32 perdió alimentación a media medición; en la tabla solo están las corridas completas.

¿Cómo afecta la confirmación de QoS 1 al loop() de la ESP32?
En nuestro sistema hay que distinguir dos loops. El de la ESP32 de la cochera no se ve afectado en absoluto: solo escribe el JSON en el buffer de 2 KB del UART y sigue, sin saber si la red está bien o mal. Durante las corridas degradadas la puerta siguió funcionando con la misma rapidez.

El loop del gateway sí lo sintió. Con red normal su vuelta más larga fue de 8 a 13 ms; con la red degradada llegó a 2.3 s (QoS 1) y 4.1 s (QoS 0), porque publish() se quedó esperando al socket. Si la ESP32 de la cochera publicara directamente con un cliente síncrono como el PubSubClient de la guía, esos segundos de bloqueo congelarían la rampa del servo y retrasarían la revisión del ultrasónico que reabre la puerta si aparece un obstáculo. Por eso, en un nodo que controla un actuador conviene publicar desde otra tarea o desde otro microcontrolador, como hicimos.

Experimento 2 y gestión de hilos
En el cliente del navegador las tramas llegaron una cada 491 a 511 ms, con 734 a 741 bytes cada una, sin que el cliente las pidiera: el servidor las empuja en cuanto llegan por MQTT. Es la diferencia con HTTP, donde el cliente tendría que preguntar una y otra vez.

En FastAPI conviven dos mundos: el hilo de paho-mqtt y el Event Loop de asyncio. El callback de MQTT nunca hace await ni toca SQLite; solo usa call_soon_threadsafe para dejar el mensaje en una cola asyncio. Así el hilo de red nunca se bloquea y el Event Loop procesa los mensajes en orden. Si se desconectan todos los clientes WebSocket, el backend sigue recibiendo, validando y guardando porque la suscripción MQTT no depende de ellos; solo deja de repartir. Un cliente que no responde en 5 s se elimina, y uno lento solo pierde sus mensajes más viejos sin frenar a los demás.

Articulación con la literatura IEEE

Bharani y Parthipan (2026) proponen una arquitectura jerárquica para monitoreo industrial en tiempo real: nodos con microcontrolador que adquieren y preprocesan datos en el borde, un gateway que los publica con un protocolo ligero como MQTT, una plataforma que los recibe y un tablero que los muestra [1]. El punto central es que ninguna capa se conecta directamente con otra: todas hablan con un intermediario. SmartGate reproduce esa estructura a escala de laboratorio: la ESP32 es el nodo que filtra en el borde, la ESP32-C3 es el gateway, Mosquitto es el intermediario Pub/Sub, FastAPI es la plataforma y la app Android es el tablero.

En nuestro caso el desacoplamiento no fue solo una buena práctica: resolvió un problema real. Como la ESP32 de la cochera no puede encender su radio, pudimos agregar el gateway C3 sin cambiar nada

del backend ni de la app; ambos siguen suscritos al mismo topic y validan el mismo JSON. De la misma forma, la app, el cliente WebSocket del navegador y el dashboard web se conectan y desconectan sin que la cochera se entere. Y si el broker o la red fallan, la puerta sigue abriendo y cerrando porque el control vive en el nodo, que es justamente lo que los autores buscan al procesar en el borde.

Los autores también señalan que la confiabilidad de entrega tiene un costo en latencia y en recursos del nodo. El experimento 1 lo muestra con datos propios: QoS 1 agrega el intercambio de PUBACK y, con la red degradada, reintentos. En nuestra arquitectura ese costo lo absorbe el gateway, no el nodo que controla la puerta, lo que confirma la ventaja de separar adquisición y comunicación en capas distintas.

Problemas y soluciones

Problema	Causa	Solución
La ESP32 no puede usar WiFi.	Brownout al encender la radio (visto en la semana 1).	Gateway ESP32-C3 conectado por UART.
La C3 no se conectaba al router.	Antena de la SuperMini con la potencia de fábrica.	Potencia de transmisión en 8.5 dBm.
Latencia de ~1.5 s en el backend.	El reloj de la laptop iba adelantado respecto a NTP.	Se midió el desfase con w32tm y se corrigió en el análisis.
La app decía «No hay coche dentro» con el carro estacionado.	El carro queda fuera del haz del ToF.	Se usa también la entrada registrada en el ciclo (arrival).
La app recibía datos de nodos de prueba.	El stream enviaba todos los nodos.	Filtro ?node_id=cochera-01 en /ws/telemetria.

Repositorio del equipo

URL: https://github.com/DiabolicAnt/cochera_IoT

Carpeta	Contenido
firmware/SmartGate_ESP32_Cochera	Nodo: sensores, filtro, control de la puerta y JSON por UART.
firmware/SmartGate_C3_Gateway	Gateway WiFi/MQTT con QoS configurable.
backend	FastAPI, modelos Pydantic, pruebas y configuración de Mosquitto.
app_movil	Proyecto Android Studio de la app SmartGate.
docs/practica2	Este reporte y las evidencias (consolas y capturas).

Conclusiones

La cadena completa funciona con la maqueta real: la ESP32 mide y controla la puerta, el gateway publica por MQTT, FastAPI valida y guarda cada mensaje y lo empuja por WebSocket, y la app avisa en el teléfono cuando la cochera se abre, queda abierta y se cierra. En condiciones normales no se perdió ningún paquete y los mensajes llegaron cada 500 ms con poco jitter.

La principal lección fue el valor del middleware: gracias al broker pudimos esquivar una limitación de hardware agregando una placa sin tocar el resto del sistema, y el control de la puerta quedó aislado de los problemas de red.

Referencias

[1] M. Bharani y V. Parthipan, «Edge-Enabled Smart Data Acquisition Using Industrial IoT Connectivity for Real-Time Monitoring Applications», en 2026 International Conference on Computational Robotics, Testing and Engineering Evaluation (ICCRTEE), 2026, pp. 1-6, doi: 10.1109/ICCRTEE68719.2026.11566569.
[2] Espressif Systems, «ESP-MQTT», ESP-IDF Programming Guide, 2026.
[3] FastAPI, «WebSockets» y «Lifespan Events», documentación oficial, 2026.
[4] Eclipse Foundation, «Paho MQTT Python Client» y «Mosquitto», documentación oficial, 2026.
[5] Android Developers, «Foreground services» y «Notifications», documentación oficial, 2026.